RTD_SIM Digital Twin Completion Roadmap

From Current State to Real-Time Adaptive System

🎯 Current State (Phase 5 Complete)

✅ What We Have:

- **System Dynamics**: Continuous ODE integration (r, K, feedback loops)
- **BDI Agents**: Static planning (desires → beliefs → intentions → actions)
- **Policy Engine**: Trigger-based interventions (threshold detection)
- **Analytics**: SHAP analysis, sensitivity analysis, report generation
- **Visualization**: Real-time dashboard with animation controls

❌ What's Missing for Real-Time Digital Twin:

- **Streaming updates**: SD still batch-computed per step
- **Event-driven architecture**: No pub-sub event bus
- **Dynamic replanning**: Agents don't adapt to perturbations
- **Real-time inputs**: No external data streams (MQTT, IoT)
- **Spatial indexing**: No efficient radius queries for agent perception
- **Collective forecasting**: No agent-to-agent information propagation
- **Hybrid transitions**: No discrete event triggers from continuous state

---

📋 Remaining Phases

Phase 6: Event-Driven Architecture (Foundation)

Goal: Replace batch step loop with event bus + streaming SD

6.1 - Event Bus Core

- [ ] Implement Redis pub/sub for multi-process communication
- [ ] Define event types: `PolicyChange`, `InfrastructureFailure`, `WeatherEvent`, `AgentModeSwitch`
- [ ] Create event schema with spatial metadata (lat/lon, radius)
- [ ] Build event publisher/subscriber pattern

6.2 - Spatial Indexing

- [ ] Implement R-tree for agent position indexing
- [ ] Efficient radius queries: "Find agents within 5km of event"
- [ ] Update on agent movement (dynamic spatial index)

6.3 - Streaming System Dynamics

- [ ] Convert SD from batch ODE to incremental updates
- [ ] Implement Kalman-like data assimilation for sensor fusion
- [ ] Event-triggered SD updates (not time-stepped)
- [ ] Hybrid state machine: continuous → discrete transitions

Files to Create:

```
rtd_sim/
├── events/
│   ├── event_bus.py         # Redis pub/sub wrapper
│   ├── event_types.py       # PolicyChange, InfraFailure, etc.
│   └── spatial_index.py     # R-tree for radius queries
├── streaming/
│   ├── sd_streaming.py      # Incremental SD updates
│   ├── kalman_filter.py     # Data assimilation
│   └── hybrid_transitions.py # Continuous → discrete
```

Estimated Time: 3-4 weeks

Phase 7: Dynamic BDI Replanning

Goal: Agents perceive events and adapt plans in real-time

7.1 - Event Perception

- [ ] Agent perception radius (e.g., 5km)
- [ ] Subscribe agents to relevant event channels
- [ ] Filter events by spatial proximity + agent type

7.2 - Belief Revision

- [ ] Update beliefs when events perceived
- [ ] Track belief certainty/staleness
- [ ] Conflict resolution (multiple sources)

7.3 - Dynamic Replanning

- [ ] Trigger replanning on belief change
- [ ] Recompute routes when infrastructure fails

- [ ] Switch modes when policies change feasibility
- [ ] Abort plans when better alternatives discovered

7.4 - Plan Monitoring

- [ ] Execution monitoring (plan vs actual)
- [ ] Failure detection (blocked route, unavailable mode)
- [ ] Rollback to safe state on failure

Files to Modify/Create:

```
agent/
├── bdi_agent.py          # Add perception + replanning
├── belief_revision.py     # Update beliefs from events
├── plan_monitor.py        # Detect execution failures
└── replan_triggers.py     # When to replan
```

Estimated Time: 2-3 weeks

Phase 8: Real-Time Data Integration

Goal: Ingest external data streams (MQTT, IoT, APIs)

8.1 - MQTT Bridge

- [ ] Connect to MQTT broker (e.g., Mosquitto)
- [ ] Subscribe to IoT topics (traffic, weather, charging)
- [ ] Parse sensor data → events
- [ ] Handle connection failures, reconnection

8.2 - Data Assimilation

- [ ] Fuse sensor data with SD state (Kalman filter)
- [ ] Detect anomalies (sensor vs model divergence)
- [ ] Correct SD state based on ground truth

8.3 - External API Integration

- [ ] Weather APIs (OpenWeatherMap, Met Office)
- [ ] Traffic APIs (Google, TomTom)
- [ ] Grid data (National Grid API)
- [ ] Charging station status (ChargePoint, Zap-Map)

8.4 - Simulated Real-Time Mode

- [ ] Replay historical data at real-time speed
- [ ] Synthetic event injection for testing
- [ ] Fast-forward/rewind capabilities

Files to Create:

```
rtd_sim/
├── integrations/
│   ├── mqtt_bridge.py       # MQTT → Event bus
│   ├── weather_api.py       # Fetch weather data
│   ├── traffic_api.py       # Fetch traffic data
│   └── grid_api.py          # Fetch grid data
├── streaming/
│   ├── data_fusion.py       # Sensor + model fusion
│   └── anomaly_detection.py # Detect divergence
```

Estimated Time: 3-4 weeks

Phase 9: Collective Forecasting

Goal: Agents share information and predict emergent scenarios

9.1 - Agent Communication

- [ ] Peer-to-peer messaging (via event bus)
- [ ] Information diffusion (gossip protocol)
- [ ] Trust/reputation system (filter bad info)

9.2 - Local Predictions

- [ ] Each agent predicts local future (next 30 min)
- [ ] Share predictions with neighbors
- [ ] Aggregate predictions → collective forecast

9.3 - Hotspot Detection

- [ ] Predict congestion (many agents → same location)
- [ ] Predict charging overload (demand > capacity)
- [ ] Predict cascade effects (failure propagation)

9.4 - Adaptive Behavior

- [ ] Avoid predicted hotspots (route around)
- [ ] Delay trips to avoid congestion
- [ ] Coordinate with other agents (implicit cooperation)

Files to Create:

```
agent/
├── communication.py      # Agent messaging
├── prediction.py         # Local forecasts
├── aggregation.py        # Collective forecasting
└── coordination.py       # Implicit cooperation
```

Estimated Time: 2-3 weeks

Phase 10: Interactive Real-Time Testing Interface

Goal: Test system with live sliders for real-world perturbations

10.1 - Real-Time Control Panel

- [ ] Sliders for continuous parameters:
  - Grid capacity (MW)
  - Fuel prices (£/L)
  - Weather severity (0-10)
  - Policy stringency (0-1)
- [ ] Buttons for discrete events:
  - Infrastructure failure (select location)
  - Policy announcement (select type)
  - Weather event (storm, heatwave)
- [ ] Inject events into live simulation

10.2 - Live Monitoring Dashboard

- [ ] Real-time SD state visualization
- [ ] Agent state heatmap (positions, modes)

- [ ] Event log (recent events + impact)
- [ ] Performance metrics (replanning rate, failures)

10.3 - Response Testing

- [ ] Inject perturbation → measure response time
- [ ] Count agents affected (within radius)
- [ ] Track replanning cascades
- [ ] Measure system stability (convergence)

10.4 - Scenario Recording

- [ ] Record slider movements + outcomes
- [ ] Replay scenarios for analysis
- [ ] Export event logs for post-mortem

Files to Create:

```
ui/
├── control_panel.py         # Real-time sliders/buttons
├── live_dashboard.py        # Real-time monitoring
├── perturbation_injector.py  # Event injection
└── scenario_recorder.py      # Record/replay
```

Estimated Time: 2-3 weeks

Phase 11: Hybrid State Machine

Goal: Discrete transitions from continuous state thresholds

11.1 - Threshold Monitors

- [ ] Watch SD stocks for threshold crossings
- [ ] Trigger discrete events on crossing
- [ ] Hysteresis to prevent oscillation

11.2 - System Mode Transitions

- [ ] Normal → Emergency (grid_util > 0.85)
- [ ] Low → High adoption (EV > 0.3)
- [ ] Stable → Cascade (failure propagation)

11.3 - Mode-Specific Behavior

- [ ] Emergency mode: restrict charging, shed load
- [ ] High adoption: unlock new infrastructure
- [ ] Cascade mode: isolate failures, reroute

11.4 - State History

- [ ] Track mode transitions over time
- [ ] Identify unstable transitions (oscillations)
- [ ] Analyze time spent in each mode

Files to Create:

```
system_dynamics/
├── threshold_monitors.py     # Watch for crossings
├── mode_transitions.py       # System state machine
├── mode_behaviors.py         # Mode-specific rules
└── transition_history.py     # Log transitions
```

Estimated Time: 2 weeks

Phase 12: Performance & Scalability

Goal: Optimize for real-time performance with 1000+ agents

12.1 - Profiling & Bottlenecks

- [ ] Profile event bus throughput
- [ ] Measure replanning latency
- [ ] Identify spatial query performance

12.2 - Optimization

- [ ] Parallel agent updates (multiprocessing)
- [ ] Batch spatial queries
- [ ] Cache route computations
- [ ] Lazy evaluation for non-critical updates

12.3 - Load Testing

- [ ] Simulate 10,000 agents
- [ ] Inject 100 events/sec
- [ ] Measure latency distribution
- [ ] Identify breaking points

12.4 - Distributed Architecture (Optional)

- [ ] Multi-node deployment (Redis cluster)
- [ ] Load balancing across workers
- [ ] Fault tolerance (node failures)

Estimated Time: 2-3 weeks

🎯 Phase 10 in Detail: Interactive Testing Interface

Since you specifically asked about sliders for real-time testing, here's the detailed design:

Control Panel Layout:

```
╔═══════════════════════════════════════════════════════════════════╗
║ 🎛️ REAL-TIME CONTROL PANEL                                        ║
╠═══════════════════════════════════════════════════════════════════╣
║                                                                   ║
║ 🌍 ENVIRONMENTAL PARAMETERS                                       ║
║ ┌─────────────────────────────────────────────────────────────┐   ║
║ │ Grid Capacity:    [▓▓▓
```

```
╔═══════════════════════════════════════════════════════════════╗
║                                                               ║
║ 📊 LIVE RESPONSE METRICS                                      ║
║ ┌─────────────────────────────────────────────────────────┐   ║
║ │ Agents Affected:     47 (within 5km radius)             │   ║
║ │ Replanning Rate:     23 agents/sec                      │   ║
║ │ Avg Response Time:   0.12 seconds                       │   ║
║ │ System Stability:    ▓▓▓
```

4. Scenario Recording:

- Save slider timeline + events
- Replay for analysis or demos
- Export to file for sharing

📅 Estimated Timeline

Phase	Duration	Dependencies
Phase 6: Event Architecture	3-4 weeks	None
Phase 7: Dynamic Replanning	2-3 weeks	Phase 6
Phase 8: Real-Time Data	3-4 weeks	Phase 6
Phase 9: Collective Forecasting	2-3 weeks	Phase 7
Phase 10: Interactive Testing	2-3 weeks	Phase 6, 7
Phase 11: Hybrid Transitions	2 weeks	Phase 6
Phase 12: Performance	2-3 weeks	All above

Total: ~16-22 weeks (4-5.5 months)

Parallel work possible:

- Phase 8 (data integration) can run parallel with Phase 7 (replanning)
- Phase 10 (testing UI) can start once Phase 6-7 complete

🎯 Priority Order (Suggested)

Critical Path:

1. **Phase 6** → Foundation (event bus + spatial index)
2. **Phase 7** → Core capability (dynamic replanning)
3. **Phase 10** → Testing infrastructure (validate 6+7 work)
4. **Phase 11** → Hybrid dynamics (key differentiator)

Can Be Deferred:

- Phase 8 (real-time data) → use synthetic events initially
- Phase 9 (collective forecasting) → nice-to-have, not critical
- Phase 12 (performance) → optimize once stable

🔬 Testing Strategy for Phase 10

Test Scenarios:

Scenario 1: Infrastructure Failure

```
1. Start simulation (normal operation)
2. Slider: Grid capacity 400 → 200 MW
3. Observe:
   - How many agents affected?
   - Do they switch from EV to diesel?
   - How long to converge?
4. Inject event: Charging station failure at lat/lon
5. Observe:
   - Do nearby EV agents reroute?
   - Do they switch modes?
   - Cascade effects?
```

Scenario 2: Policy Shock

```
1. Slider: Carbon tax £0 → £100/tonne
2. Observe:
   - Mode shift diesel → EV?
   - Replanning rate spike?
   - SD adoption flow increase?
3. Button: Announce "EV mandate 2030"
4. Observe:
   - Long-term planning changes?
   - Infrastructure investment signals?
```

Scenario 3: Weather Event

```
1. Slider: Weather severity 0 → 9 (storm)
2. Observe:
   - Agents delay trips?
   - Switch to public transit?
   - Route around affected areas?
3. Inject: Road closure (select on map)
4. Observe:
   - Dynamic rerouting?
   - Congestion cascade?
```

💡 Key Design Decisions

1. Event Bus: Redis vs Kafka?

Recommendation: Redis

- Simpler setup (single binary)
- Sufficient throughput (<10k events/sec)
- Built-in pub/sub + spatial extensions
- Can upgrade to Kafka later if needed

2. Spatial Index: R-tree vs Grid?

Recommendation: R-tree (rtree library)

- Efficient radius queries O(log n)
- Dynamic updates (agent movement)
- Standard library, well-tested

3. SD Integration: Pull vs Push?

Recommendation: Push (event-driven)

- Agents publish mode switches → SD updates immediately
- No polling overhead
- More responsive to perturbations

4. Replanning: Eager vs Lazy?

Recommendation: Hybrid

- Eager for critical events (infrastructure failure)
- Lazy for minor changes (small price adjustments)
- Threshold-based triggering

🎉 What Success Looks Like

At the end of Phase 12, RTD_SIM will be a **true real-time digital twin**:

✅ **Perceive:** Agents receive events from pub-sub bus (MQTT, Redis) ✅ **Deliberate:** Update beliefs, trigger replanning on threshold ✅ **Adapt:** Recompute routes, switch modes, adjust schedules ✅ **Interact:** Share predictions with peers, coordinate implicitly ✅ **Forecast:** Collective predictions of congestion, overload, cascades

Interactive Testing:

- Drag sliders → instant system response
- Click map → inject spatial event → watch ripple effect
- Record scenarios → replay → analyze

Real-Time Capable:

- 1000+ agents updating in real-time
- <100ms event propagation latency
- <500ms replanning latency per agent
- Stable under perturbations (no oscillations)

This is an ambitious but achievable roadmap! 🚀